

Cours complet C & C++

De l'algorithmique aux objets

S1-S2 – Licence Informatique – UAPV

Arthur

2025 / 2026

Table des matières

Première partie

Bases de la programmation en C

1 Algorithmes, variables, conditions et boucles

1.1 C'est quoi un algorithme ?

Un algorithme est une **suite finie d'instructions**, exécutées dans l'ordre, qui prend des entrées et produit un résultat. Un bon algorithme est **peu complexe**, **rapide** et **économe en ressources**.

Exemple

Deux façons de représenter un algorithme :

- **Pseudo-code** : description textuelle proche du langage humain
- **Algorithme** : représentation graphique avec des boîtes et des flèches

1.2 C vs Python bas niveau vs haut niveau

TABLE 1 : Comparaison C / Python

	C / C++	Python
Niveau	Bas (proche machine)	Haut (proche humain)
Gestion mémoire	Manuelle	Automatique
Compilation	Obligatoire (gcc / g++)	Interprété
Performance	Très rapide	Plus lent
Portabilité	Bonne	Excellente

Le compilateur (gcc pour le C, g++ pour le C++) transforme le code source en langage machine, effectue des analyses lexicale, syntaxique et sémantique, signale les erreurs et émet des avertissements.

```

1 # Compilation
2 gcc programme.c -o programme
3
4 # Exécution
5 ./programme

```

Listing 1: Compilation et exécution d'un programme C

1.3 Structure minimale d'un programme C

```

1 #include <stdio.h>
2
3 int main() {
4     printf("Hello World\n");
5     return 0;

```

6 }

Listing 2: Hello World en C

- `#include <stdio.h>` : inclut la bibliothèque d'entrées/sorties standard
- `int main()` : point d'entrée, retourne 0 en cas de succès
- `printf` : affiche du texte dans le terminal
- Chaque instruction se termine par ;

1.4 Les variables

Une variable possède : un **nom**, un **type**, une **valeur** et une **adresse** en mémoire.

1.4.1 Déclaration et initialisation

```

1 // Déclaration
2 int age;
3 double taille;
4 bool vivant;
5
6 // Initialisation (affectation)
7 age = 20;
8 taille = 1.78;
9 vivant = true; // nécessite #include <stdbool.h> en C
10
11 // Déclaration + initialisation en une ligne
12 int score = 100;
13 double prix_ht = 9.99;
```

Listing 3: Déclaration et initialisation de variables

1.4.2 Types principaux

TABLE 2 : Types de base en C

Type	Taille	Description	Exemple
int	32 bits	Entier	<code>int x = 42;</code>
double	64 bits	Réel double prec.	<code>double pi = 3.14;</code>
float	32 bits	Réel simple prec.	<code>float f = 2.5f;</code>
char	8 bits	Caractère	<code>char c = 'A';</code>
bool	8 bits	Booléen (C99)	<code>bool ok = true;</code>

1.4.3 Convention de nommage (snake_case)

- En anglais, en minuscules
- Commence par une lettre de l'alphabet
- Séparateur : `_` (ex : `prix_final`, `length_array`)
- Pas de mots-clés réservés (`int`, `return`, etc.)

1.4.4 Entrée / sortie : printf et scanf

```

1  #include <stdio.h>
2
3  int main() {
4      int age;
5      double taille;
6
7      // Affichage
8      printf("Bonjour !\n");
9      printf("Score : %d\n", 42);           // %d = int
10     printf("Pi = %f\n", 3.14159);        // %f = double
11     printf("Lettre : %c\n", 'A');        // %c = char
12
13     // Saisie utilisateur
14     printf("Entrez votre age : ");
15     scanf("%d", &age);                   // & obligatoire !
16
17     printf("Entrez votre taille : ");
18     scanf("%lf", &taille);               // %lf pour double
19
20     printf("Vous avez %d ans et mesurez %f m\n", age, taille);
21     return 0;
22 }
```

Listing 4: Entrées et sorties en C

Règle

Correspondance format / type pour `printf` et `scanf` :

- `%d` – int
- `%f` – double (`printf`) / `%lf` – double (`scanf`)
- `%c` – char
- `%p` – adresse mémoire (pointeur)
- Toujours mettre `&` devant la variable dans `scanf` !

1.4.5 Portée des variables

Une variable n'existe que dans le **bloc** (`{}`) où elle est déclarée, et dans les blocs imbriqués. Dès que l'on sort du bloc, la variable est détruite.

```

1  int main() {
2      int a = 3;
3
4      if (a < 5) {
5          int c = 5;    // c existe seulement ici
6      }
7
8      // printf("%d\n", c); // ERREUR : c n'existe plus
9      return 0;
10 }
```

Listing 5: Portée locale

1.5 Les conditions

```
1 int doigt = 2;
2
3 if (doigt == 1) {
4     printf("THUMB UP !\n");
5 } else if (doigt == 2) {
6     printf("ON MONTRE PAS DU DOIGT !\n");
7 } else if (doigt == 3) {
8     printf("SOIS POLI !\n");
9 } else if (doigt == 4 || doigt == 5) {
10    printf("ILS SERVENT A RIEN\n");
11 } else {
12    printf("C'est quoi cette tentacule ?!\n");
13 }
```

Listing 6: Conditions if / else if / else

1.5.1 Opérateurs relationnels

Opérateur	Signification
==	Égal
!=	Différent
<	Inférieur
<=	Inférieur ou égal
>	Supérieur
>=	Supérieur ou égal

1.5.2 Opérateurs logiques

Opérateur	Signification
&&	ET logique
	OU logique
!	NON (inverse la valeur booléenne)

```
1 if (a > 0 && b > 0)                // a ET b positifs
2 if (a < 0 || b > 0)                // a négatif OU b positif
3 if (a > 0 && (b > 0 || c > 0))    // a positif ET (b OU c
    positifs)
4 if (a > 0 && !(b > 0 || c > 0)) // a positif ET NI b NI c
    positifs
```

Listing 7: Exemples d'opérateurs logiques

1.6 Les boucles

1.6.1 Boucle for nombre d'itérations connu

```

1 // for (initialisation ; condition de continuation ; pas)
2 for (int i = 0; i < 10; i = i + 1) {
3     printf("%d\n", i);
4 }
```

Listing 8: Boucle for

1.6.2 Boucle while nombre d'itérations inconnu

```

1 int i = 0;
2 while (i < 10) {
3     i = chiffreAuHasard();
4 }
```

Listing 9: Boucle while

Règle

- **for** : quand on connaît à l'avance le nombre d'itérations.
- **while** : quand on répète jusqu'à ce qu'une condition soit satisfaite.

2 Les fonctions

2.1 Qu'est-ce qu'une fonction ?

Une fonction est une **boîte qui encapsule un algorithme** : elle prend des entrées (paramètres), effectue un traitement et retourne un résultat. Elle permet de ne pas répéter du code, de le rendre lisible et de le réutiliser.

2.2 Déclaration et définition

```

1 // prototype : type_retour nom(type param1, type param2)
2 int additionner(int nombre1, int nombre2)
3 {
4     int result = nombre1 + nombre2;
5     return result;
6 }
7
8 int main() {
9     int a = 12, b = 18;
10    int somme = additionner(a, b); // appel
11    printf("Somme : %d\n", somme);
12    return 0;
13 }
```

Listing 10: Anatomie d'une fonction

Règle

Convention de nommage des fonctions (camelCase) :

- Commence par un verbe, en minuscule : `calculerMoyenne`, `afficherScore`
- Chaque nouveau mot commence par une majuscule
- Pas de symboles spéciaux

2.3 Fonctions sans retour : void

```

1 void afficherSalutation(int age) {
2     if (age >= 18) {
3         printf("Bienvenue !\n");
4     } else {
5         printf("Interdit aux mineurs.\n");
6     }
7     // pas de return nécessaire
8 }
```

Listing 11: Fonction void

2.4 Passage par valeur (copie)

En C, les paramètres sont **toujours passés par valeur** : la fonction reçoit une **copie** de la variable. Modifier le paramètre dans la fonction ne modifie pas la variable originale.

```

1 void addOne(int a) {
2     a = a + 1; // modifie la COPIE
3 }
4
5 int main() {
6     int a = 15;
7     addOne(a);
8     printf("%d\n", a); // affiche 15, pas 16 !
9     return 0;
10 }
```

Listing 12: Passage par valeur la variable originale n'est pas modifiée

Important

En Python, le passage par défaut est le **passage par référence** pour les objets mutables. C'est l'inverse du C !

2.5 Retourner un résultat

`return` renvoie une valeur **et quitte immédiatement** la fonction.

```

1 int addOne(int number) {
2     number = number + 1;
3     return number; // retourne la valeur modifiée
4 }
```

```

5
6 // return quitte la fonction immédiatement :
7 bool isAboveTen(int number) {
8     if (number > 10) {
9         return true; // on sort ici si vrai
10    }
11    return false;     // équivalent à l'else
12 }

```

Listing 13: Retour de valeur et return early

2.6 La pile (stack) et les fonctions

À chaque appel de fonction, une nouvelle **frame** est empilée sur la pile. Les variables locales de cette fonction y vivent. À la sortie de la fonction, la frame est dépilée automatiquement et les variables sont détruites.

```

1 void c() { int z = 3; }
2 void b() { int y = 2; }
3 void a() { int x = 1; b(); }
4
5 int main() {
6     int w = 3;
7     if (w == 3) {
8         int v = 5; // portée locale au if
9         a();
10    }
11    c();
12    return 0;
13 }
14 // Pile au moment de b() :
15 //   b()   : y
16 //   a()   : x
17 //   if    : v
18 //   main  : w

```

Listing 14: Pile d'exécution

2.7 Fichiers .h et .c

Pour organiser un projet, on sépare les **déclarations** (fichier .h) des **définitions** (fichier .c).

```

1 #ifndef MATHOPERATION_H
2 #define MATHOPERATION_H
3
4 int addOne(int a);
5 double sum(double a, double b);
6 int power(int a, int pow);
7
8 #endif

```

Listing 15: mathematicOperations.h

```
1 #include "mathematicOperations.h"
2
3 int addOne(int a) {
4     a = a + 1;
5     return a;
6 }
7
8 double sum(double a, double b) {
9     double result = a + b;
10    return result;
11 }
```

Listing 16: mathematicOperations.c

```
1 #include <stdio.h>
2 #include "mathematicOperations.h"
3
4 int main() {
5     int result = addOne(12);
6     printf("Résultat : %d\n", result);
7     return 0;
8 }
```

Listing 17: main.c

3 Les pointeurs

3.1 Rappel : la mémoire

Toute variable possède une **adresse** (numéro de case mémoire) et un **type** (qui dit combien de bits lire). Un pointeur stocke justement cette adresse.

TABLE 3 : Exemple de représentation mémoire

Variable	Adresse	Valeur
a	0x7ffcc0bee04c	3
pAge	0x9ffdce1f45ee	0x7ffcc0bee04c

3.2 Déclarer et utiliser un pointeur

```
1 int a = 3;
2
3 // Déclaration d'un pointeur (convention : nom commence par p)
4 int *pAge;
5
```

```

6 // Récupérer l'adresse de a et la stocker dans pAge
7 pAge = &a;
8
9 // Afficher l'adresse et la valeur pointée
10 printf("%d\n", a);           // 3
11 printf("%p\n", pAge);       // 0x7ffcc...
12 printf("%d\n", *pAge);      // 3 (déréférencement)
13
14 // Modifier la valeur via le pointeur
15 *pAge = 6;
16 printf("%d\n", a);          // 6 (a est modifié !)
17 printf("%d\n", *pAge);      // 6

```

Listing 18: Déclaration, et *

Règle

- & devant une variable : donne son **adresse**
- * à la **déclaration** : indique que la variable est un pointeur
- * à l'**utilisation** : accède à la valeur pointée (déréférencement)

3.3 Erreurs courantes

TABLE 4 : Erreurs fréquentes avec les pointeurs

Erreur	Cause
Null Pointer Exception	Pointeur non initialisé, mis à NULL ou désalloué
Invalid Pointer Error	Adresse modifiée par erreur, ou espace mémoire désalloué

Important

Toujours initialiser un pointeur ! Si vous ne pointez sur rien encore, mettez-le à NULL :

```
1 int *pAge = NULL;
```

3.4 Pointeurs comme paramètres (in-out)

Le passage par pointeur permet de **modifier la variable originale** depuis une fonction ce que le passage par valeur ne permet pas.

```

1 void addOne(int *pBase) {
2     *pBase = *pBase + 1;
3 }
4
5 int main() {
6     int a = 3;

```

```

7   addOne(&a);           // on passe l'ADRESSE de a
8   printf("%d\n", a);    // affiche 4 !
9   return 0;
10 }
```

Listing 19: Passage par pointeur modifie l'original

Astuce

En C, si une fonction doit retourner **plusieurs résultats**, on passe des pointeurs en paramètres (paramètres in-out). En C++, on utilisera les références pour simplifier cette syntaxe.

3.5 Pointeur de pointeur

```

1  int a = 42;
2  int *pB = &a;
3  int **pPC = &pB; // pointeur vers un pointeur
4
5  printf("%p\n", pPC); // adresse de pB
6  printf("%p\n", *pPC); // adresse de a
7  printf("%d\n", **pPC); // valeur de a : 42
```

Listing 20: Double pointeur

4 Les tableaux

4.1 Motivation

Plutôt que 300 variables séparées pour stocker les notes de 300 étudiants, un tableau permet de tout regrouper en une seule variable et de tout traiter avec une boucle.

4.2 Déclaration et initialisation

```

1  // Déclaration (taille fixe à la compilation)
2  int tab[300];
3
4  // Initialisation totale
5  int tab5[5] = {3, 5, 8, 12, 8};
6  double tabD[3] = {3.5, 8.2, 7.3};
7  bool tabB[4] = {0, 1, 1, 1};
8
9  // Initialisation partielle (le reste vaut 0)
10 int tab2[5] = {3, 5, 8}; // -> {3, 5, 8, 0, 0}
11
12 // Initialisation après déclaration (case par case)
13 int tab3[5];
14 tab3[0] = 0;
15 tab3[1] = 2;
```

```

16 // ou avec une boucle :
17 for (int i = 0; i < 5; i++) {
18     tab3[i] = i * 2;
19 }

```

Listing 21: Déclaration et initialisation de tableaux

4.3 Accès aux données

La numérotation commence à 0. `tab[i]` accède à la case d'indice `i` (décalage de `i` depuis le début du tableau).

```

1 int tab[4] = {3, 8, 56, 158};
2
3 printf("%d\n", tab[0]); // 3   (1ère case)
4 printf("%d\n", tab[1]); // 8   (2ème case)
5 printf("%d\n", tab[3]); // 158 (4ème case)
6
7 tab[3] = 14;           // modifie la 4ème case

```

Listing 22: Accès et modification

4.4 Tableau et pointeur

Un tableau se comporte comme un **pointeur constant** vers sa première case. La variable `tab` contient l'adresse du premier élément.

```

1 int tab[4] = {3, 8, 56, 158};
2
3 printf("%p\n", tab);           // adresse de tab[0]
4 printf("%d\n", *tab);          // 3   (équivalent à tab[0])
5 printf("%d\n", *(tab + 1));    // 8   (équivalent à tab[1])

```

Listing 23: Équivalence tableau / pointeur

4.5 Taille statique vs taille réelle

```

1 #define MAX_ETUDIANTS 2000
2
3 int etudiants[MAX_ETUDIANTS];
4 int tailleMax = MAX_ETUDIANTS;
5 int tailleReelle;
6
7 printf("Nombre d'étudiants cette année : ");
8 scanf("%d", &tailleReelle);
9
10 for (int i = 0; i < tailleReelle; i++) {
11     scanf("%d", &etudiants[i]);
12 }

```

Listing 24: Gérer taille max et taille réelle

Important

La taille d'un tableau statique est fixée à la compilation. On ne peut pas écrire `int tab[n]`; si `n` est saisi par l'utilisateur (avant C99). La solution propre : réserver un max et stocker la taille réelle séparément.

4.6 Tableaux et fonctions

Un tableau passé à une fonction l'est comme un pointeur : les modifications dans la fonction impactent le tableau original.

```

1 // Toujours passer la taille avec le tableau !
2 void affiche(int tab[], int taille) {
3     for (int i = 0; i < taille; i++) {
4         printf("%d\n", tab[i]);
5     }
6 }
7
8 void saisieTableau(int tableau[], int taille) {
9     for (int i = 0; i < taille; i++) {
10        scanf("%d", &tableau[i]);
11    }
12 }
13
14 int main() {
15     int tab[4] = {2, 5, 8, 9};
16     affiche(tab, 4);
17     return 0;
18 }
```

Listing 25: Passage de tableau en paramètre

4.7 Tableaux multidimensionnels

```

1 // Déclaration : int tab[lignes][colonnes]
2 int tab[3][4];
3
4 // Initialisation
5 int mat[3][4] = { {0, 1, 2, 3},
6                  {4, 5, 6, 7},
7                  {8, 9, 10, 11} };
8
9 // Parcours avec boucles imbriquées
10 for (int i = 0; i < 3; i++) {
11     for (int j = 0; j < 4; j++) {
12         printf("%d ", mat[i][j]);
13     }
14     printf("\n");
15 }
16
17 // Passage en fonction
```

```
18 void saisieTab2D(int tableau[][4], int tailleX, int tailleY);
```

Listing 26: Tableau 2D

5 Les structures

5.1 Motivation

Quand plusieurs variables décrivent la même entité (nom, date de naissance, numéro de sécu d'une personne), il devient difficile de les gérer séparément. Les structures permettent de les **regrouper en un seul type**.

5.2 Définir une structure

Les structures se définissent dans un fichier `.h` portant leur nom.

```
1 #ifndef TAMAGOTCHI_H
2 #define TAMAGOTCHI_H
3
4 typedef struct {
5     int     sante;
6     int     energie;
7     double  faim;
8     double  moral;
9 } Tamagotchi;
10
11 #endif
```

Listing 27: Tamagotchi.h

Règle

Règles de nommage d'une structure :

- Commence par une majuscule, c'est un **nom** (pas un verbe)
- Pas de `_` ni de caractères spéciaux
- N'oubliez pas le `;` après la fermeture `}`

5.3 Utiliser une structure

```
1 #include "Tamagotchi.h"
2 #include <stdio.h>
3
4 int main() {
5     // Déclaration
6     Tamagotchi poupi;
7
8     // Accès et modification : NomInstance.NomChamp
9     poupi.sante = 12;
10    poupi.energie = 80;
11}
```

```

12     printf("Sant   : %d\n", poupi.sante);
13
14     // Initialisation s  quentielle    la d  claration
15     Tamagotchi riri = {100, 100, 0.0, 100.0};
16
17     // Initialisation s  lective (le reste vaut 0)
18     Tamagotchi loulou = {.energie = 100, .moral = 100.0};
19
20     // Tableau de structures
21     Tamagotchi tamadex[151];
22     tamadex[0].sante = 24;
23
24     return 0;
25 }

```

Listing 28: D  claration, initialisation et acc  s

5.4 Fonctions et structures

```

1  #include "Tamagotchi.h"
2  #include <stdio.h>
3
4  // Passage par copie : les modifications n'impactent pas l'
   original
5  void afficherSante(Tamagotchi tama) {
6      printf("Sant   : %d\n", tama.sante);
7  }
8
9  // Retourner une structure modifi  e
10 Tamagotchi setSante(Tamagotchi tama, int sante) {
11     if      (sante < 0)      tama.sante = 0;
12     else if (sante > 100)    tama.sante = 100;
13     else                    tama.sante = sante;
14     return tama;
15 }
16
17 // Passage par pointeur : modifie l'original directement
18 void setSantePtr(Tamagotchi *pTama, int sante) {
19     if      (sante < 0)      (*pTama).sante = 0;
20     else if (sante > 100)    (*pTama).sante = 100;
21     else                    (*pTama).sante = sante;
22 }

```

Listing 29: Tamagotchi.c associer des fonctions    une structure

Astuce

`(*pTama).sante` et `pTama->sante` sont   quivalents. L'op  rateur `->` est une notation raccourcie tr  s utilis  e en C pour acc  der    un champ via un pointeur.

```

1 int main() {
2     Tamagotchi poupi = {100, 100, 0.0, 100.0};
3
4     // Passage par copie + récupération du résultat
5     poupi = setSante(poupi, 150);    // sera clampé à 100
6     afficherSante(poupi);
7
8     // Passage par pointeur
9     setSantePtr(&poupi, -5);          // sera clampé à 0
10    printf("Santé : %d\n", poupi.sante);
11
12    return 0;
13 }

```

Listing 30: Utilisation dans main.c

6 Allocation dynamique

6.1 Pourquoi allouer dynamiquement ?

Les variables locales vivent sur la **pile** et sont détruites en sortant de leur bloc. L'**allocation dynamique** permet de créer de la mémoire sur le **tas** (heap), dont la durée de vie n'est pas liée à la portée des variables.

TABLE 5 : Pile (Stack) vs Tas (Heap)

	Pile (Stack)	Tas (Heap)
Gestion	Automatique	Manuelle
Durée de vie	Liée au bloc	Jusqu'à <code>free()</code>
Taille	Limitée (~8 Mo)	Quasi-illimitée
Vitesse	Rapide	Plus lent

6.2 malloc allouer de la mémoire

```

1 #include <stdlib.h>
2
3 // Allouer un entier sur le tas
4 int *pVar = malloc(sizeof(int));
5 *pVar = 3;
6 printf("%d\n", *pVar);    // 3
7
8 // sizeof retourne la taille en octets d'un type
9 // sizeof(int) -> 4, sizeof(double) -> 8, etc.

```

Listing 31: Allocation avec malloc

Important

Toujours stocker l'adresse renvoyée par `malloc` dans un pointeur ! Sinon l'espace est alloué mais introuvable.

6.3 free libérer la mémoire

```

1 int *pAge = malloc(sizeof(int));
2 *pAge = 3;
3
4 // Libération
5 free(pAge);
6 pAge = NULL; // Bonne pratique : mettre à NULL après free

```

Listing 32: Libération avec free

Important

Fuite mémoire : si on perd l'adresse avant d'avoir fait `free`, la mémoire reste allouée jusqu'à la fin du programme. Chaque `malloc` doit avoir son `free`.

6.4 Tableau alloué dynamiquement

```

1 int n;
2 printf("Taille du tableau : ");
3 scanf("%d", &n);
4
5 // Allouer n entiers contigus
6 int *arr = malloc(n * sizeof(int));
7
8 // Utilisation identique à un tableau classique
9 arr[5] = 3;
10 arr[0] = 10;
11
12 // Libération
13 free(arr);
14 arr = NULL;

```

Listing 33: Tableau de taille choisie à l'exécution

6.5 malloc et structures

```

1 // Structure avec un tableau interne
2 typedef struct {
3     int     sante;
4     double *stats;
5 } Tamagotchi;
6
7 int main() {

```

```

8 // Allocation de la structure
9 Tamagotchi *t = malloc(sizeof(Tamagotchi));
10 (*t).sante = 100;
11
12 // Allocation du tableau interne
13 int n = 5;
14 (*t).stats = malloc(n * sizeof(double));
15 (*t).stats[0] = 9.5;
16
17 // Libération (ordre inverse de l'allocation)
18 free((*t).stats);
19 free(t);
20 t = NULL;
21
22 return 0;
23 }

```

Listing 34: Allocation dynamique d'une structure

Règle

Ordre de libération : toujours libérer les allocations **imbriquées en premier**, puis la structure principale. L'ordre inverse de l'allocation.

Deuxième partie

Programmation en C++

7 Introduction au C++

7.1 C++ par rapport au C

Le C++ est une **extension du C** qui ajoute la programmation orientée objet, les références, la surcharge de fonctions, les templates, et une bibliothèque standard riche. Tout code C valide est (presque) valide en C++.

TABLE 6 : Principales différences C / C++

Fonctionnalité	C	C++
Entrées/sorties	printf / scanf	cout / cin
Allocation dynamique	malloc / free	new / delete
Références	Non	Oui (&)
Classes et objets	Non (struct seulement)	Oui
Surcharge de fonctions	Non	Oui
Passage par référence	Par pointeur	Par référence (&)

7.2 Caractéristiques du langage

Le C++ est un langage **compilé**, c'est-à-dire qu'avant d'être exécuté, le code source doit être traduit en langage machine par un compilateur.

Exemple

Processus de compilation et exécution :

1. Écriture du code dans `programme.cpp`
2. Compilation : `g++ programme.cpp -o programme`
3. Exécution : `./programme`

7.3 Structure minimale d'un programme

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     cout << "Hello World!" << endl;
6     return 0;
7 }
```

Listing 35: Programme minimal en C++

- `#include <iostream>` : inclut la bibliothèque d'entrées/sorties
- `using namespace std;` : permet d'utiliser `cout` et `cin` sans préfixe
- `int main()` : fonction principale, point d'entrée du programme

7.4 Entrées et sorties

En C++, on utilise `cout` et `cin` à la place de `printf` et `scanf`.

```
1 int age;
2 string nom;
3
4 // Affichage
5 cout << "Entrez votre nom : ";
6 cin >> nom;
7
8 cout << "Entrez votre age : ";
9 cin >> age;
10
11 cout << "Bonjour " << nom << ", vous avez " << age << " ans."
    << endl;
```

Listing 36: Utilisation de `cout` et `cin`

Règle

Opérateurs de flux :

- « : pour envoyer vers la sortie (`cout`)
- » : pour récupérer depuis l'entrée (`cin`)
- `endl` : retour à la ligne

8 Les types de base et les tableaux

8.1 Types de données

TABLE 7 : Types de données en C++

Type	Description	Exemple
<code>int</code>	Entier	<code>int x = 42;</code>
<code>double</code>	Réel double précision	<code>double pi = 3.14;</code>
<code>float</code>	Réel simple précision	<code>float f = 2.5f;</code>
<code>char</code>	Caractère	<code>char c = 'A';</code>
<code>bool</code>	Booléen	<code>bool ok = true;</code>
<code>string</code>	Chaîne de caractères	<code>string s = "hello";</code>

8.2 Déclaration de tableaux

```

1 // Tableau a une dimension
2 int tab[10];           // Tableau de 10 entiers
3 tab[0] = 5;           // Premier element (indice 0)
4 tab[9] = 12;          // Dernier element (indice 9)
5
6 // Tableau a deux dimensions
7 double matrice[3][4]; // 3 lignes, 4 colonnes
8 matrice[0][0] = 1.5;
9
10 // Initialisation lors de la declaration
11 int nombres[] = {10, 20, 30, 40, 50};

```

Listing 37: Tableaux statiques

Important

Les tableaux statiques ont une taille fixe définie à la compilation. Cette taille ne peut pas être modifiée pendant l'exécution du programme.

9 Pointeurs et références

9.1 Les pointeurs

Un pointeur est une variable qui contient l'**adresse mémoire** d'une autre variable.

```

1 int x = 42;
2 int* p;           // p est un pointeur sur int
3
4 p = &x;           // p contient l'adresse de x
5
6 cout << x;        // Affiche : 42
7 cout << &x;       // Affiche : 0x7fff5a3b524 (adresse de x)
8 cout << p;        // Affiche : 0x7fff5a3b524 (meme adresse)
9 cout << *p;       // Affiche : 42 (valeur pointee)

```

Listing 38: Déclaration et utilisation de pointeurs

Règle

Les deux opérateurs clés :

- `&` : opérateur d'adresse, donne l'adresse d'une variable
- `*` : opérateur de déréférencement, accède à la valeur pointée

9.2 Schéma mémoire

Variable	Adresse	Valeur
x	0x1000	42
p	0x2000	0x1000

Le pointeur p (stocké à l'adresse 0x2000) contient l'adresse de x (0x1000).

9.3 Les références

Une référence est un **alias** pour une variable existante. C'est la grande nouveauté du C++ par rapport au C.

```

1 int x = 10;
2 int& ref = x;    // ref est une reference sur x
3
4 ref = 20;        // Modifie x
5 cout << x;       // Affiche : 20

```

Listing 39: Utilisation des références

Astuce

Une référence doit être initialisée lors de sa déclaration et ne peut plus changer de cible par la suite.

TABLE 8 : Comparaison Pile vs Tas

	Pile (Stack)	Tas (Heap)
Taille	Limitée (~8 Mo)	Quasi-illimitée
Libération	Automatique	Manuelle (<code>delete</code>)
Utilisation	Variables locales	Allocation dynamique
Rapidité	Rapide	Plus lent

10 Allocation dynamique

10.1 La pile (Stack) et le tas (Heap)

10.2 Allocation dans la pile

```

1 void fonction() {
2     int x = 5;           // Cree dans la pile
3     double tab[10];      // Cree dans la pile
4     // ...
5 } // x et tab sont automatiquement detruits ici

```

Listing 40: Variables locales dans la pile

10.3 Allocation dans le tas avec `new/delete`

En C++, on remplace `malloc/free` par `new/delete` qui sont plus sûrs et gèrent les constructeurs.

```

1 // Allocation d'un seul element
2 int* p = new int;
3 *p = 42;
4 delete p;           // Libere la memoire
5
6 // Allocation d'un tableau
7 int n = 100;
8 int* tableau = new int[n];
9 tableau[0] = 10;
10 tableau[99] = 20;
11 delete[] tableau;   // ATTENTION aux crochets !

```

Listing 41: Allocation dynamique avec `new`

Important

Règle d'or :

- Chaque `new` doit avoir son `delete`
- Chaque `new[]` doit avoir son `delete[]`
- Sinon : fuite mémoire !

10.4 Exemple complet

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int taille;
6     cout << "Entrez la taille du tableau : ";
7     cin >> taille;
8
9     // Allocation dynamique
10    int* tab = new int[taille];
11
12    // Remplissage
13    for (int i = 0; i < taille; i++) {
14        tab[i] = i * 10;
15    }
16
17    // Affichage
18    for (int i = 0; i < taille; i++) {
19        cout << tab[i] << " ";
20    }
21    cout << endl;
22
23    // Liberation
24    delete[] tab;
25
26    return 0;
27 }
```

Listing 42: Gestion dynamique d'un tableau

11 Passage de paramètres

11.1 Passage par valeur

Le paramètre reçoit une **copie** de la variable.

```
1 void modifier(int x) {
2     x = 10;           // Modifie la COPIE
3 }
4
5 int main() {
6     int a = 5;
7     modifier(a);
8     cout << a;        // Affiche : 5 (non modifie)
9 }
```

Listing 43: Passage par valeur

11.2 Passage par référence

Le paramètre est un **alias** de la variable originale.

```

1 void modifier(int& x) {    // Le & est crucial
2     x = 10;                // Modifie l'ORIGINAL
3 }
4
5 int main() {
6     int a = 5;
7     modifier(a);
8     cout << a;            // Affiche : 10 (modifie !)
9 }

```

Listing 44: Passage par référence

11.3 Passage par référence constante

Permet d'éviter la copie tout en protégeant la variable.

```

1 void afficher(const int& x) {
2     cout << x;
3     // x = 10;            // ERREUR de compilation
4 }

```

Listing 45: Passage par référence constante

Règle

Quand utiliser quoi ?

- **Par valeur** : types simples (int, char, bool, double)
- **Par référence constante** (const &) : objets en lecture seule
- **Par référence** (&) : objets à modifier

11.4 Tableau comparatif

TABLE 9 : Modes de passage de paramètres

Mode	Copie ?	Modifiable ?	Syntaxe
Par valeur	Oui	Non (copie)	void f(int x)
Par référence	Non	Oui	void f(int& x)
Par réf. const.	Non	Non	void f(const int& x)
Par pointeur (C)	Non	Oui	void f(int* x)

12 Programmation orientée objet

12.1 Concepts fondamentaux

La **Programmation Orientée Objet** (POO) permet de :

- Regrouper données et fonctions dans une même structure (classe)
- Protéger les données (encapsulation)

- Créer des types personnalisés
- Améliorer la maintenabilité et la réutilisabilité du code

Astuce

Le lien avec les structures C est direct : une **classe** en C++ est comme une **struct** en C, mais avec des **méthodes intégrées** et un contrôle d'accès (**private** / **public**).

12.2 Structure d'une classe

```

1  #ifndef POINT_H
2  #define POINT_H
3
4  class Point {
5  private:      // Donnees membres privees
6      double x;
7      double y;
8
9  public:       // Fonctions membres publiques
10     // Constructeur
11     Point(double abs, double ord);
12
13     // Methodes
14     void afficher() const;
15     void deplacer(double dx, double dy);
16
17     // Accesseurs (getters)
18     double getX() const;
19     double getY() const;
20
21     // Mutateurs (setters)
22     void setX(double valeur);
23     void setY(double valeur);
24 };
25
26 #endif

```

Listing 46: Déclaration d'une classe - fichier Point.h

12.3 Implémentation

```

1  #include "Point.h"
2  #include <iostream>
3  using namespace std;
4
5  // Constructeur
6  Point::Point(double abs, double ord) {
7      x = abs;
8      y = ord;
9  }

```

```

10
11 void Point::afficher() const {
12     cout << "(" << x << ", " << y << ")" << endl;
13 }
14
15 void Point::deplacer(double dx, double dy) {
16     x += dx;
17     y += dy;
18 }
19
20 double Point::getX() const { return x; }
21 double Point::getY() const { return y; }
22 void Point::setX(double valeur) { x = valeur; }
23 void Point::setY(double valeur) { y = valeur; }

```

Listing 47: Implémentation de la classe - fichier Point.cpp

12.4 Utilisation

```

1 #include "Point.h"
2 #include <iostream>
3 using namespace std;
4
5 int main() {
6     Point p1(3.0, 5.0);
7     Point p2(1.0, 2.0);
8
9     p1.afficher();           // (3, 5)
10    p1.deplacer(2.0, 1.0);
11    p1.afficher();           // (5, 6)
12
13    cout << "x = " << p1.getX() << endl;
14
15    p1.setX(10.0);
16    p1.afficher();           // (10, 6)
17
18    // p1.x = 20;           // ERREUR : donnee privée
19
20    return 0;
21 }

```

Listing 48: Programme principal - fichier main.cpp

Règle

Principe d'encapsulation :

- Les données membres sont **private**
- Les méthodes d'interface sont **public**
- On accède aux données via des accesseurs (getters) et mutateurs (setters)

13 Constructeurs et destructeurs

13.1 Le constructeur

Le constructeur est une fonction appelée **automatiquement** lors de la création d'un objet.

Règle

Caractéristiques :

- Porte le même nom que la classe
- N'a pas de type de retour
- Sert à initialiser l'objet
- Peut effectuer des allocations dynamiques

13.2 Types de constructeurs

```

1 class Tableau {
2 private:
3     int* tab;
4     int taille;
5
6 public:
7     // 1) Constructeur par défaut
8     Tableau() {
9         taille = 10;
10        tab = new int[10];
11    }
12
13    // 2) Constructeur avec parametres
14    Tableau(int n) {
15        taille = n;
16        tab = new int[n];
17    }
18
19    // 3) Constructeur avec valeur par défaut
20    // Tableau(int n = 10) { taille = n; tab = new int[n]; }
21 };

```

Listing 49: Différents types de constructeurs

13.3 Utilisation

```

1 int main() {
2     Tableau t1;           // Appelle le constructeur par défaut
3     Tableau t2(20);       // Appelle le constructeur avec n=20
4     Tableau t3(50);       // Appelle le constructeur avec n=50
5
6     // ATTENTION : ceci ne cree PAS un objet !
7     // Tableau t4();      // Declare une fonction !

```

```
8 }
```

Listing 50: Appel des constructeurs

13.4 Le destructeur

Le destructeur est une fonction appelée **automatiquement** lors de la destruction d'un objet.

Règle

Caractéristiques :

- Porte le nom de la classe précédé de ~
- N'a pas de paramètres ni de type de retour
- Sert à libérer les ressources (mémoire, fichiers, etc.)

```
1 class Tableau {
2 private:
3     int* tab;
4     int taille;
5
6 public:
7     Tableau(int n) {
8         taille = n;
9         tab = new int[n];
10        cout << "Construction d'un tableau de " << n << endl;
11    }
12
13    ~Tableau() {
14        delete[] tab;
15        cout << "Destruction du tableau" << endl;
16    }
17 };
```

Listing 51: Destructeur

14 Constructeur par copie et opérateur d'affectation

14.1 Problème de la copie superficielle

Quand une classe contient un pointeur, la copie par défaut ne copie que l'adresse les deux objets pointent alors sur la même zone mémoire. Cela provoque des erreurs lors de la destruction.

```
1 class Tableau {
2     int* tab;
3     int taille;
4 public:
5     Tableau(int n) { taille = n; tab = new int[n]; }
```

```
6     ~Tableau() { delete[] tab; }
7 };
8
9 int main() {
10     Tableau t1(5);
11     Tableau t2 = t1;    // Copie superficielle !
12     // t1 et t2 partagent le MEME tableau
13     // A la destruction : double delete -> crash !
14 }
```

Listing 52: Problème de copie superficielle

14.2 Constructeur par recopie

```
1 // Dans la classe :
2 Tableau(const Tableau& autre) {
3     taille = autre.taille;
4     tab = new int[taille];    // Nouvelle allocation
5     for (int i = 0; i < taille; i++) {
6         tab[i] = autre.tab[i];    // Copie des valeurs
7     }
8 }
```

Listing 53: Constructeur par recopie (copie profonde)

14.3 Opérateur d'affectation

```
1 Tableau& operator=(const Tableau& autre) {
2     if (this == &autre)    // Auto-affectation
3         return *this;
4
5     delete[] tab;    // Libere l'ancienne
6     memoire
7
8     taille = autre.taille;
9     tab = new int[taille];
10    for (int i = 0; i < taille; i++) {
11        tab[i] = autre.tab[i];
12    }
13
14    return *this;
15 }
```

Listing 54: Opérateur d'affectation

Règle

Règle des trois : si vous définissez l'un des trois suivants, définissez les trois :

1. Destructeur
2. Constructeur par copie
3. Opérateur d'affectation

15 Listes chaînées

15.1 Principe

Une liste chaînée est une structure de données où chaque élément (**maillon** ou **nud**) contient une valeur et un **pointeur vers le maillon suivant**.

```

1 class Maillon {
2 public:
3     int valeur;
4     Maillon* suivant;
5
6     Maillon(int v) : valeur(v), suivant(nullptr) {}
7 };

```

Listing 55: Structure d'un maillon

15.2 Classe ListeChaine

```

1 class ListeChaine {
2 private:
3     Maillon* tete;
4
5 public:
6     ListeChaine() : tete(nullptr) {}
7
8     ~ListeChaine() {
9         Maillon* courant = tete;
10        while (courant != nullptr) {
11            Maillon* suivant = courant->suivant;
12            delete courant;
13            courant = suivant;
14        }
15    }
16
17    // Ajouter en tete
18    void ajouterEnTete(int valeur) {
19        Maillon* nouveau = new Maillon(valeur);
20        nouveau->suivant = tete;
21        tete = nouveau;
22    }
23

```

```

24 // Parcourir et afficher
25 void afficher() const {
26     Maillon* courant = tete;
27     while (courant != nullptr) {
28         cout << courant->valeur << " -> ";
29         courant = courant->suivant;
30     }
31     cout << "NULL" << endl;
32 }
33
34 bool estVide() const { return tete == nullptr; }
35 };

```

Listing 56: Liste chaînée complète

15.3 Comparaison tableau / liste chaînée

TABLE 10 : Tableau vs Liste chaînée

	Tableau	Liste chaînée
Taille	Fixe	Dynamique
Accès à l'élément i	O(1) (direct)	O(n) (parcours)
Insertion en tête	O(n)	O(1)
Mémoire	Continue	Fragmentée

16 Exercices récapitulatifs

16.1 Exercice 1 : Classe Rectangle

Énoncé : Créer une classe `Rectangle` avec longueur et largeur, un constructeur, et les méthodes `aire()`, `perimetre()` et `afficher()`.

Solution :

```

1 #ifndef RECTANGLE_H
2 #define RECTANGLE_H
3
4 class Rectangle {
5 private:
6     double longueur;
7     double largeur;
8
9 public:
10     Rectangle(double l, double L);
11     double aire() const;
12     double perimetre() const;
13     void afficher() const;
14 };
15

```

```
16 #endif
```

Listing 57: Rectangle.h

```
1 #include "Rectangle.h"
2 #include <iostream>
3 using namespace std;
4
5 Rectangle::Rectangle(double l, double L) {
6     longueur = l;
7     largeur = L;
8 }
9
10 double Rectangle::aire() const {
11     return longueur * largeur;
12 }
13
14 double Rectangle::perimetre() const {
15     return 2 * (longueur + largeur);
16 }
17
18 void Rectangle::afficher() const {
19     cout << "Rectangle " << longueur << "x" << largeur << endl;
20     cout << "Aire : " << aire() << endl;
21     cout << "Perimetre : " << perimetre() << endl;
22 }
```

Listing 58: Rectangle.cpp

16.2 Exercice 2 : Classe Vecteur dynamique

Énoncé : Créer une classe **Vecteur** qui gère un tableau dynamique d'entiers avec :

- Constructeur avec taille
- Destructeur
- Constructeur par recopie
- Opérateur d'affectation
- Méthode `get(int i)` et `set(int i, int valeur)`

Solution :

```
1 #ifndef VECTEUR_H
2 #define VECTEUR_H
3
4 class Vecteur {
5 private:
6     int* tab;
7     int taille;
8
9 public:
10     Vecteur(int n);
11     ~Vecteur();
12     Vecteur(const Vecteur& autre);
13     Vecteur& operator=(const Vecteur& autre);
```

```

14
15     int get(int i) const;
16     void set(int i, int valeur);
17     int getTaille() const;
18     void afficher() const;
19 };
20
21 #endif

```

Listing 59: Vecteur.h

```

1  #include "Vecteur.h"
2  #include <iostream>
3  using namespace std;
4
5  Vecteur::Vecteur(int n) {
6      taille = n;
7      tab = new int[n];
8      for (int i = 0; i < n; i++) tab[i] = 0;
9  }
10
11 Vecteur::~Vecteur() { delete[] tab; }
12
13 Vecteur::Vecteur(const Vecteur& autre) {
14     taille = autre.taille;
15     tab = new int[taille];
16     for (int i = 0; i < taille; i++) tab[i] = autre.tab[i];
17 }
18
19 Vecteur& Vecteur::operator=(const Vecteur& autre) {
20     if (this == &autre) return *this;
21     delete[] tab;
22     taille = autre.taille;
23     tab = new int[taille];
24     for (int i = 0; i < taille; i++) tab[i] = autre.tab[i];
25     return *this;
26 }
27
28 int Vecteur::get(int i) const {
29     if (i < 0 || i >= taille) {
30         cout << "Erreur : indice hors limites" << endl;
31         return 0;
32     }
33     return tab[i];
34 }
35
36 void Vecteur::set(int i, int valeur) {
37     if (i < 0 || i >= taille) {
38         cout << "Erreur : indice hors limites" << endl;
39         return;
40     }
41     tab[i] = valeur;

```

```

42 }
43
44 int Vecteur::getTaille() const { return taille; }
45
46 void Vecteur::afficher() const {
47     cout << "[";
48     for (int i = 0; i < taille; i++) {
49         cout << tab[i];
50         if (i < taille - 1) cout << ", ";
51     }
52     cout << "]" << endl;
53 }

```

Listing 60: Vecteur.cpp

16.3 Exercice 3 : Pile (Stack) avec liste chaînée

Énoncé : Implémenter une pile (LIFO) avec :

- `empiler(int x)` : ajoute x au sommet
- `depiler()` : retire et retourne l'élément du sommet
- `estVide()` : teste si la pile est vide
- `sommet()` : retourne l'élément du sommet sans le retirer

Solution :

```

1 class Maillon {
2     int valeur;
3     Maillon* suivant;
4     friend class Pile;
5 public:
6     Maillon(int v, Maillon* s = nullptr) : valeur(v), suivant(s) {}
7 };
8
9 class Pile {
10 private:
11     Maillon* sommetPile;
12
13 public:
14     Pile() : sommetPile(nullptr) {}
15
16     ~Pile() {
17         while (!estVide()) depiler();
18     }
19
20     void empiler(int x) {
21         sommetPile = new Maillon(x, sommetPile);
22     }
23
24     int depiler() {
25         if (estVide()) { cout << "Erreur : pile vide" << endl;
return 0; }

```

```

26     int valeur = sommetPile->valeur;
27     Maillon* temp = sommetPile;
28     sommetPile = sommetPile->suivant;
29     delete temp;
30     return valeur;
31 }
32
33 bool estVide() const { return sommetPile == nullptr; }
34
35 int sommet() const {
36     if (estVide()) { cout << "Erreur : pile vide" << endl;
return 0; }
37     return sommetPile->valeur;
38 }
39 };

```

Listing 61: Pile.h et Pile.cpp

17 Aide-mémoire

17.1 Tableau récapitulatif des opérateurs

TABLE 11 : Opérateurs importants en C / C++

Opérateur	Nom	C	C++
&	Adresse		
*	Déréférencement / pointeur		
->	Accès membre via pointeur		
.	Accès membre d'un objet		
::	Résolution de portée	—	
new	Allocation dynamique	—	
delete	Libération dynamique	—	
malloc	Allocation dynamique		()
free	Libération dynamique		()

17.2 Commandes de compilation

```

1 # Compilation C
2 gcc programme.c -o programme
3
4 # Compilation C++
5 g++ programme.cpp -o programme
6
7 # Avec debug
8 g++ -g programme.cpp -o programme
9
10 # Avec warnings

```

```
11 g++ -Wall -Wextra programme.cpp -o programme
12
13 # Plusieurs fichiers
14 g++ main.cpp Point.cpp Rectangle.cpp -o programme
15
16 # Execution
17 ./programme
18
19 # Detection de fuites memoire (Linux)
20 valgrind ./programme
```

Listing 62: Compilation C et C++

17.3 Modèle de fichiers .h et .cpp (C++)

```
1 #ifndef NOMCLASSE_H
2 #define NOMCLASSE_H
3
4 class NomClasse {
5 private:
6     // Attributs prives
7
8 public:
9     NomClasse(); // Constructeur
10    NomClasse(/* parametres */); // Constructeur
    paramétré
11    ~NomClasse(); // Destructeur
12    NomClasse(const NomClasse& autre); // Recopie
13    NomClasse& operator=(const NomClasse& autre); //
    Affectation
14
15    // Methodes publiques
16 };
17
18 #endif
```

Listing 63: Modèle de fichier .h

18 Conclusion

Ce document couvre l'ensemble du parcours C/C++ du semestre 1 et 2 de licence informatique. En résumé :

Partie C :

- Algorithmes, variables, conditions, boucles
- Fonctions : portée, passage par valeur, retour, fichiers .h/.c
- Pointeurs : &, *, passage in-out
- Tableaux statiques et lien avec les pointeurs
- Structures : regrouper des données cohérentes
- Allocation dynamique : `malloc` / `free`, tas vs pile

Partie C++ :

- Syntaxe C++ : `cout/cin`, `string`, références
- Allocation dynamique : `new` / `delete`
- Programmation orientée objet : classes, encapsulation
- Constructeurs et destructeurs
- Règle des trois : destructeur, copie, affectation
- Listes chaînées

Règle

Les trois règles d'or du C++ :

1. **Règle des trois** : Destructeur + constructeur par copie + opérateur d'affectation
2. **RAII** : Acquérir les ressources dans le constructeur, les libérer dans le destructeur
3. **const-correctness** : Utiliser `const` partout où c'est pertinent

Bon courage pour les révisions !